

6. Distributed Shared Memory

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

6.1 Allgemeines

- DSM = Distributed Shared Memory.
- Idee: Illusion eines gemeinsamen Speichers für Multicomputer → Kommunikation über den Speicher statt explizitem Message Passing
 - Erste Implementierung IVY von K. Li (1986)
 - TreadMarks (W. Zwaenepol) in Intel C++ Compiler
 - seit 2006 in OpenMP optional verfügbar



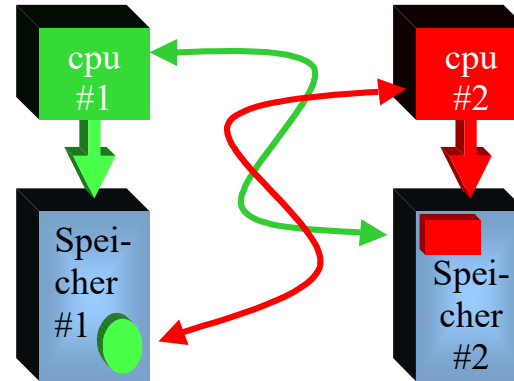
6.1 Allgemeines (2)

- Zahlreiche Projekte und Implementierungen:
 - Hardwarebasiert: Multiprozessor-Maschinen
 - CPUs teilen sich gemeinsamen Speicher
 - Softwarebasiert: Multicomputer mehrere
 - vernetzte Rechner (mit privatem Speicher)
- Neuere Entwicklungen:
 - Distributed transactional memory/caches
 - Application caches (memcached)
 - Key-Value Stores (basierend auf Infiniband)
 - Graphverarbeitung in Clustern
 - ...



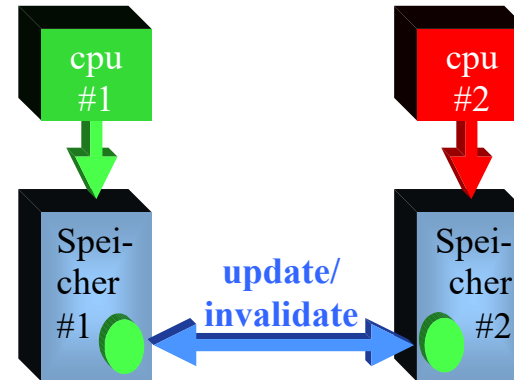
Replikation

- Datenzugriff ohne Replikation:
 - schneller Zugriff im lokalen Speicher
 - langsamer Zugriff im fremden Speicher
 - (auch ohne Replikation ist hier eine Synchronisierung erforderlich)



Replikation

- Datenzugriff mit Replikation:
 - Z.B. auf allen Kopien schreiben & von lokaler Kopie lesen
 - Vermeidet häufige Lesezugriffe über das Netz
 - Update- oder Invalidate-Protokoll notwendig



- Unterschiedliche Konsistenzmodelle
→ definieren wann eine Änderung sichtbar wird (siehe später)

6.2 Seitenbasierter DSM

- DSM als lineare Folge von Seiten (i.d.R. 4 KB)
- **Speicherzugriffserkennung erfolgt hier pro Speicherseite**
- Identische logische Adressen auf verschiedenen Knoten verweisen auf die gleichen Daten
 - Typischerweise wird auf jedem Knoten beim Start eines Prozesses ein großer logischer Adressblock reserviert
 - Dieser muss bei allen Knoten die gleiche logische Startadresse haben
 - Nur so sind Zeiger unproblematisch
 - Siehe auch UNIX shared memory mappings

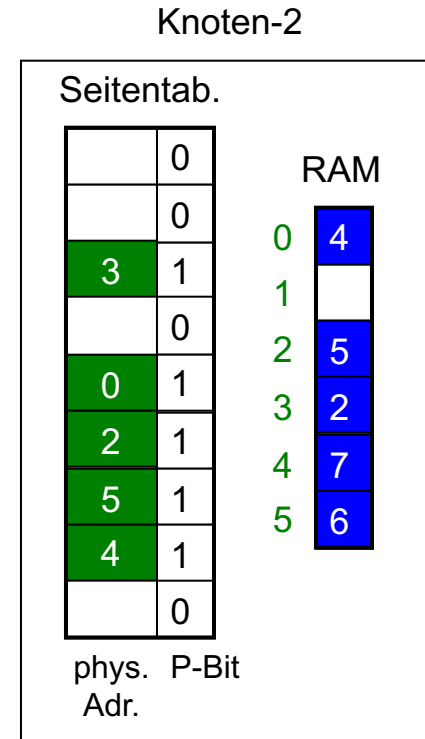
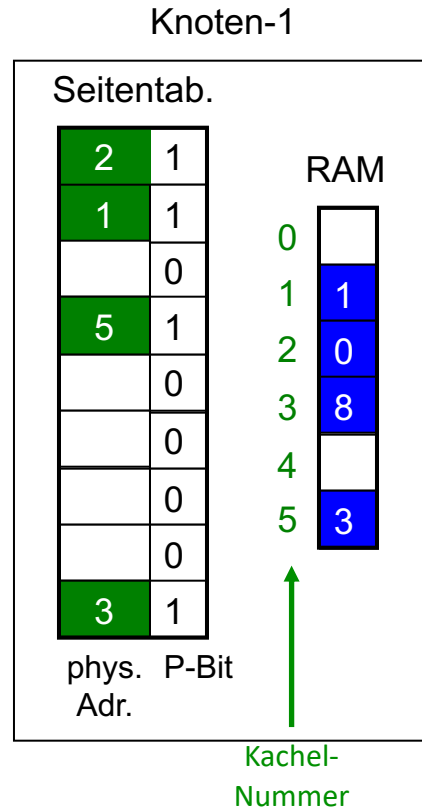
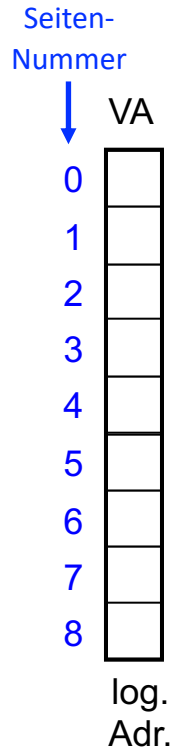


Implementierung des verteilten Zugriffs

- **Erweitern des Paging im Betriebssystem**
→ **Seiten auch über das Netz ein- und auslagern**
- Vorteil: Speicherzugriff transparent und sprachunabhängig erkennbar
- Nachteile:
 - Page-Fault ist vergleichsweise langsam
 - False-Sharing (siehe später)



Ablauf eines entfernten Speicherzugriffs

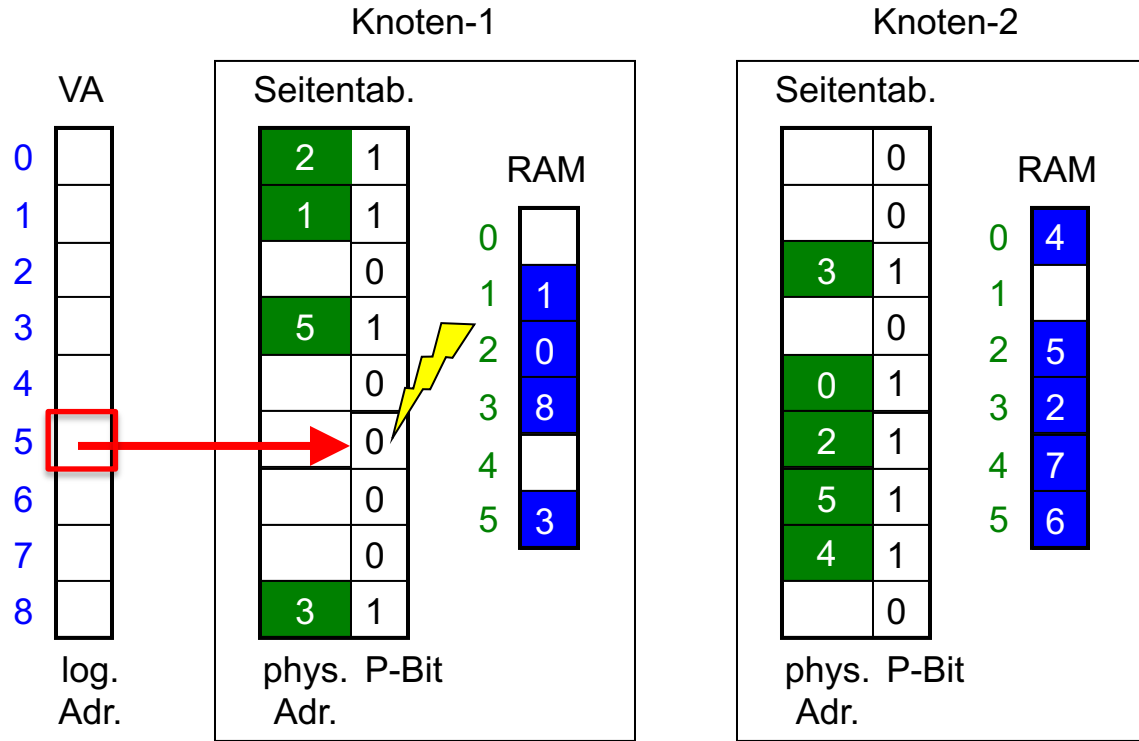


VA = virtueller Adressraum

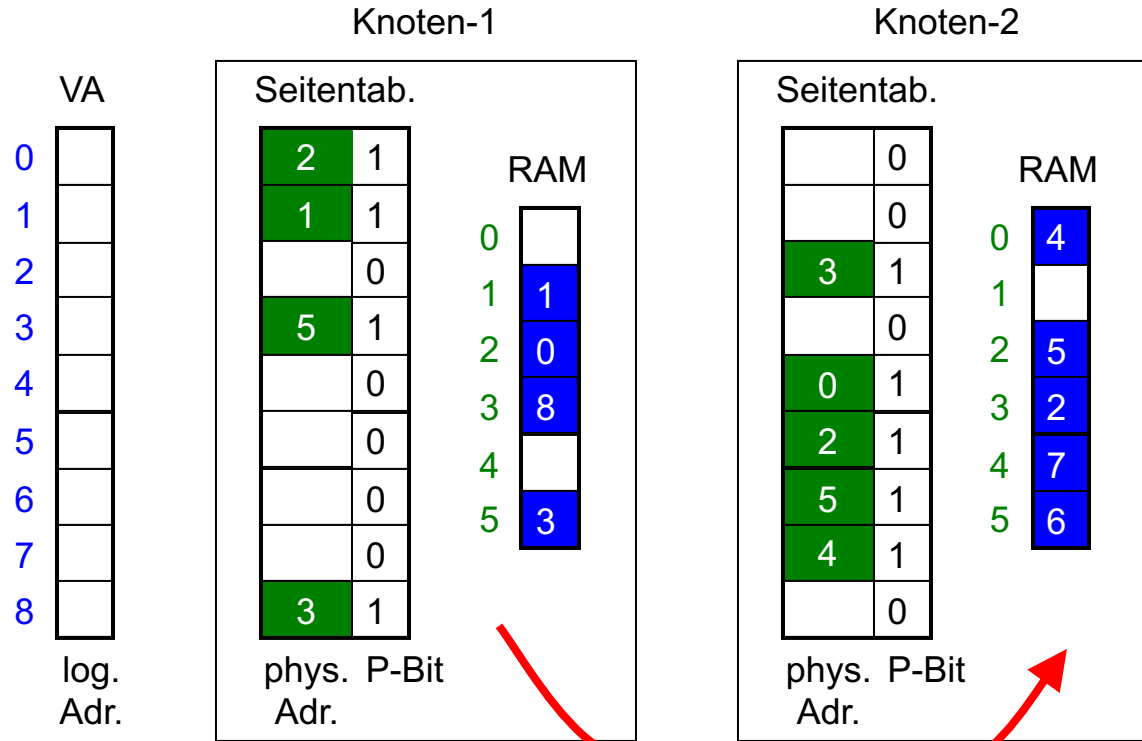
RAM = physikalischer Speicher



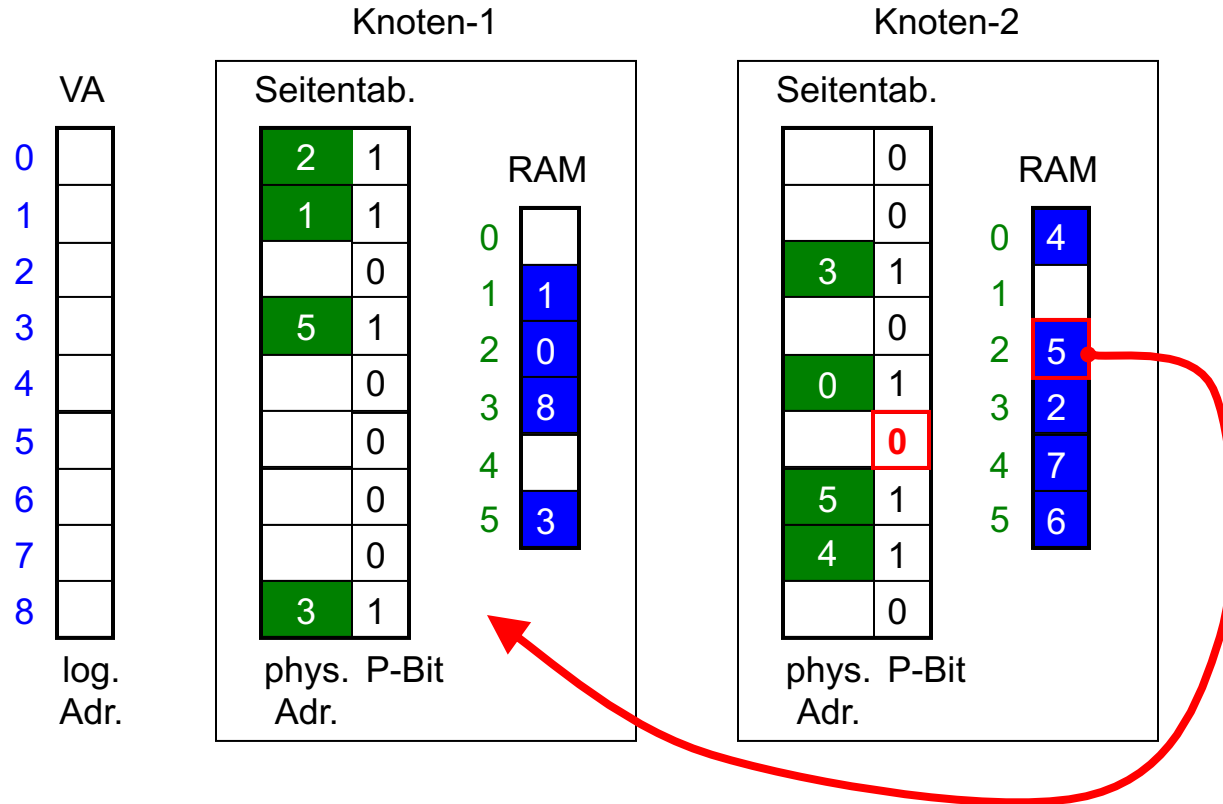
Zugriff auf Seite 5 → nicht präsent → Page Fault



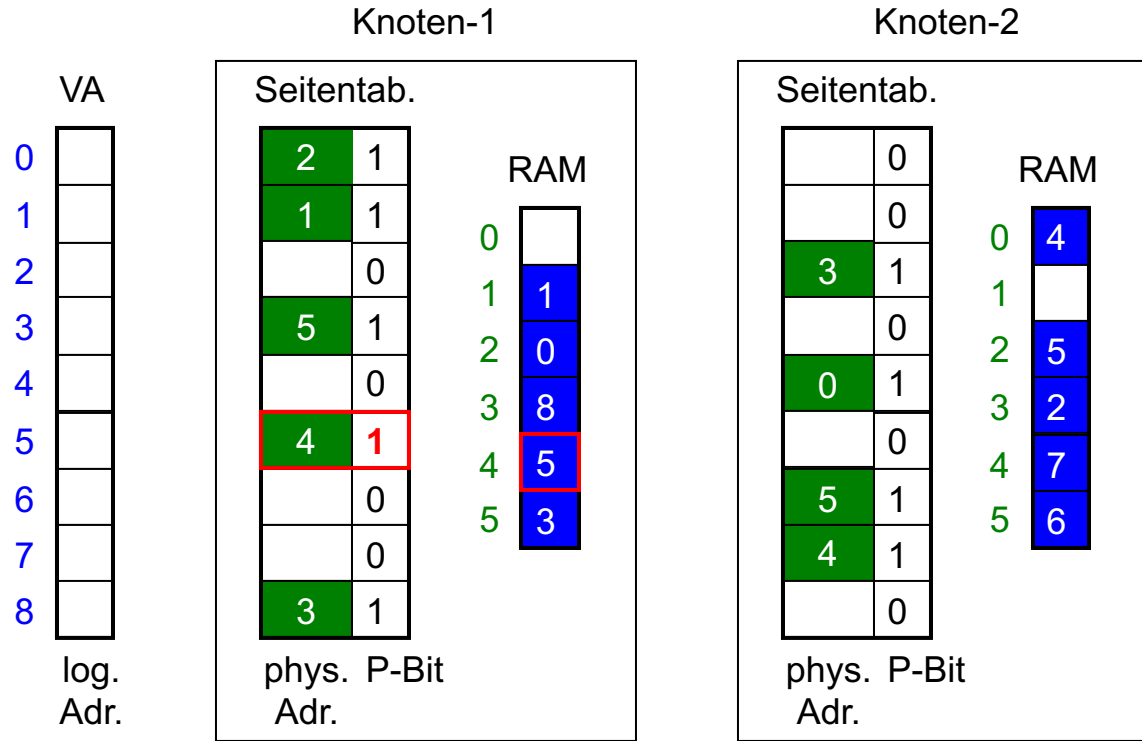
DSM sucht die Seite im Netz und fragt dann an



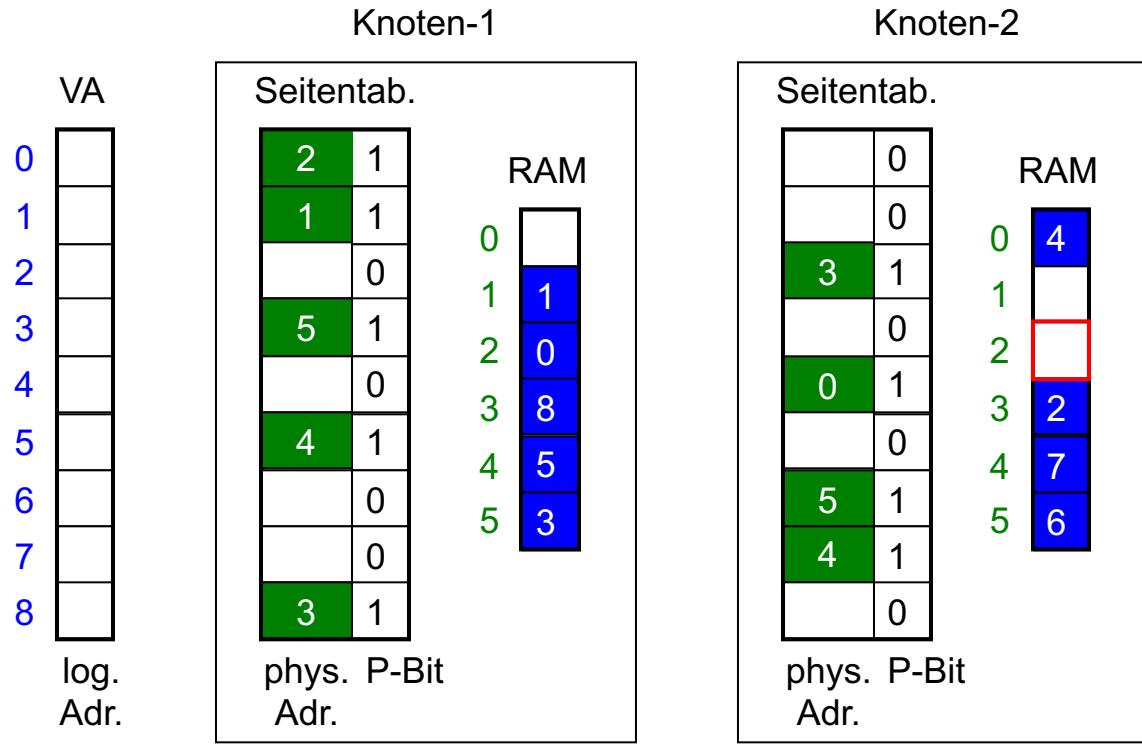
Seite auf „nicht präsent“ setzen & Inhalt versenden



Seite eintragen & Zugriff auf Knoten-1 freigeben



Kachel auf Knoten-2 freigeben



Auffinden von Seiten im Netz

- Bei einem Page-Fault muss die fehlende Seite gefunden werden
- Feste Zuordnung von Adressbereichen zu Knoten
- Anfordern per Multicast
- Verzeichnisdienst: zentral oder verteilt



True-Sharing

- Zwei oder mehr Objekte liegen auf einer Seite
- Knoten greift ein Objekt zu, wodurch eine Seite aus dem Netz geholt wird.
- Greift nun der Knoten auf weitere Objekte auf dieser Seite zu, so sind diese lokal bereits vorhanden → True-Sharing → Caching-Effekt
- Zugriffe auf benachbarte Speicheradressen sind aufgrund des Lokalisitätsprinzips wahrscheinlich (siehe Betriebssysteme).



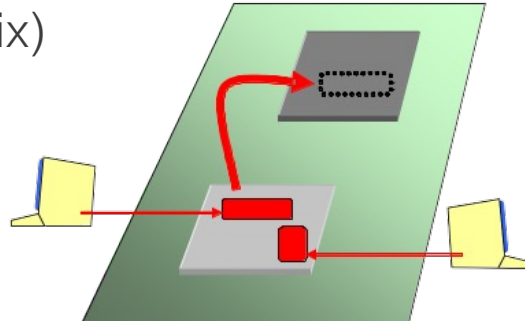
False-Sharing

- Grundphänomen in seitenbasierten DSM-Systemen:
 - ≥ 2 Objekte liegen auf derselben Speicherseite
 - Verschiedene Knoten greifen auf unterschiedl. Objekte innerhalb einer Speicherseite zu
 - Hierbei ist ≥ 1 Zugriff schreiben
 - $\rightarrow$ Objekte/Seite wird durch das Netzwerk
 - hin- und her transportiert (u.U. sehr oft $\rightarrow$ Seitenflattern)
- Wichtig: zeitlich abhängiges Phänomen.



False-Sharing

- Lösung 1: Objekte geschickt allozieren → Benutzer muss vorab Zugriffsmuster durchdenken (aufwändig & komplex)
- Lösung 2: Instanzen dynamisch relocieren
Zeigeranpassung notwendig (Plurix)



- Lösung 3: abgeschwächte Konsistenzmodelle
- Lösung 4: kleinere Sharing-Granularität (siehe nächste Folien)
→ kein True-Sharing mehr

6.3 Objekt-basierter DSM

- Basis ist eine objekt-orientierte Programmiersprache
 - Data-Sharing für Instanzvariablen (auch Referenzen); mit Vererbung, Polymorphie
- Vermeidet False-Sharing (aber auch kein True-Sharing), aber komplexe Objektstrukturen sind gemeinsam zugänglich
- **Zugriffserkennung pro Objekt:**
 - Transparent mithilfe spezieller Compiler
 - Generieren bei Zugriff auf Instanzen und Variablen entsprechende Aufrufe an die DSM-Laufzeitbibliothek
- Beispiel: Orca
 - H. Bal et.al. “Performance Evaluation of the Orca Shared Object System”, ACM Transactions on Computer Systems, Vol. 16, No. 1, Febr. 1998.



6.4 Variablen-basierter DSM

- Attribute für einzelne Variablen:
 - local, read-only, shared, synchronized
 - Spezieller Compiler berücksichtigt Attribute
- **Grundidee: nur punktuell einige Variablen gemeinsam nutzen**
 - Sehr kleine Speichergranularität → kein False Sharing, aber auch kein True Sharing
 - Programmierer muss genau überlegen, welche Variablen gemeinsam genutzt werden sollen und welche nicht
- Beispiel: Munin
 - J.B. Carter et.al.: "Implementation and performance of Munin", In Proc. of the 13th ACM Symposium on Operating Systems Principles, p.152-164, Oct. 1991.



6.5 Message Passing vs Distributed Shared Memory



Nachrichtenbasierte Kommunikation

- Serialisierung und Deserialisierung von Objekten
- Gekapselt in Middleware, z.B. gRPC, Web services, ...
- Mit umfangreichen sprachunabhängigen Laufzeitumgebungen
- Aber Konsistenz & Fehlertoleranz von Daten/Objekt ist die Aufgabe des Programmierers
- Bei korrekter und optimaler Programmierung ist die maximale Nebenläufigkeit erzielbar.



Speicherbasierte Kommunikation

- Vereinfachte implizite Kommunikation
- Serialisierung von Objekten entfällt
→ keine Hüllenbildung für Referenzen notwendig
- Konsistenz wird für alle Objekte im DSM garantiert
- Inhärente Replikation ist gute Basis für Fehlertoleranz
- I.d.R. etwas langsamer als Message Passing,
aber einfacher zu programmieren

